

LOW LEVEL DESIGN

The LLD Runbook

Ten problems, worked the way you would actually solve them.

Part 1 gave you the vocabulary. This is where it earns its keep. Each problem runs end to end in two pages — the questions worth asking, the model that follows, and the follow-up the interviewer is waiting to spring.

Attempt each one before reading the answer. The value is not in the solutions; it is in finding out where your instincts differ from them.

How to read these problems

Every problem takes exactly two pages and the two pages always do the same two jobs. Once you know the shape, you can find any part of any problem without hunting.

LEFT PAGE — THE DESIGN

- **Problem statement** as an interviewer would give it, ambiguity intact.
- **Clarifying questions** worth asking, and which ones actually change the design.
- **Requirements**, functional and non-functional, separated.
- **Entities** — the nouns that survive scrutiny.
- **Class diagram** of the model those nouns produce.

RIGHT PAGE — THE DEPTH

- **Relationships** and the **patterns** that fell out of them.
- **Code sketch** of the decisive part, with full source linked.
- **Complexity**, then the **follow-ups** to expect.
- **Alternative design** — the other defensible answer.
- **Common mistakes**, and what the interviewer is really watching.

How to actually use them

Read the problem statement and the clarifying questions, then stop. Give yourself fifteen minutes and a blank page before you read another word. What you are training is the sequence from **6.1**, and you cannot train it by reading someone else's finished answer.

When you do read on, compare structure rather than names. If your classes differ from mine but your seam is in the same place, you have the right answer. If your seam is somewhere else, work out which of us guessed the axis better — sometimes it will be you.

EVERY PROBLEM ENDS WITH THE FOLLOW-UP

That is deliberate, and it mirrors the round itself. The design is the setup; the change in requirements is where the marks are. If you only have time to study half of each problem, study the right-hand page.

P1 LRU Cache

PROBLEM STATEMENT

Design a cache with a fixed capacity that supports `get` and `put`. When the cache is full, the least recently used entry is evicted to make room.

This one usually arrives as a data structures question. It becomes a design question the moment the interviewer changes the eviction rule.

CLARIFYING QUESTIONS

- Does `get` count as a use? *Yes — and if you don't ask, ask yourself why you assumed it.*
- Is a miss an exception or a sentinel? Changes the signature, not the design.
- Fixed capacity, or resizable at runtime?
- Single-threaded? Say you're assuming so, and that you'll revisit it.
- Any expiry or time-to-live? Usually out of scope, but worth ten seconds to confirm.

FUNCTIONAL

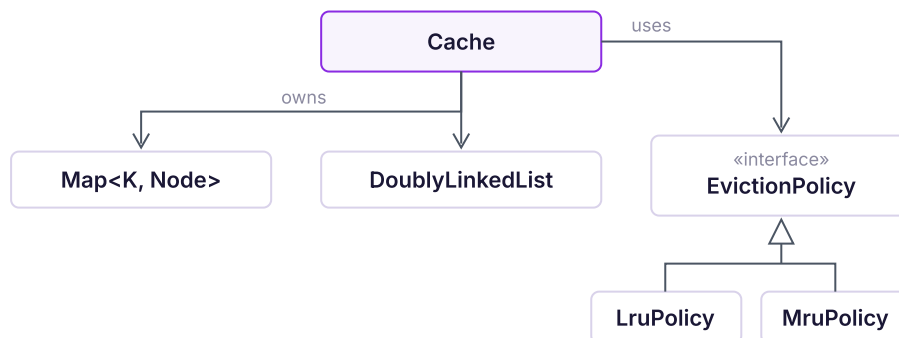
- `get(key)` returns the value and marks it most recently used.
- `put(key, value)` inserts or updates, and marks it most recently used.
- At capacity, `put` evicts the least recently used entry first.

NON-FUNCTIONAL

- Both operations in $O(1)$ — this is the whole point of the problem.
- Memory bounded by capacity, not by total keys ever seen.
- Eviction order must be exact, not approximate.

ENTITIES

`Node` holding key, value and both links · `DoublyLinkedList` maintaining recency order · a map from key to node for constant-time lookup · `Cache` coordinating the two · `EvictionPolicy`, the seam.



The map gives $O(1)$ lookup; the list gives $O(1)$ reordering. The policy decides which end to drop.

P1 LRU Cache, in depth

RELATIONSHIPS

- **Cache** owns the map and the list outright.
- The policy is handed in, not created — that is what makes it swappable.
- The policy touches the list only. It never sees the map.

PATTERNS USED

- **Strategy** for eviction — the only pattern this problem needs.
- A **Factory** only if policies come from config.

THE DECISIVE PART

```
void put(K key, V value) {
    if (map.containsKey(key)) {
        Node node = map.get(key);
        node.value = value;
        order.moveToFront(node);
        return;
    }
    if (map.size() == capacity) {
        Node victim = policy.select(order); // the only line that knows the rule
        map.remove(victim.key);           // why Node stores its key
        order.remove(victim);
    }
    map.put(key, order.addFront(new Node(key, value)));
}
```

github.com/varunkashyap-vkd/lll-runbook/lru-cache

COMPLEXITY

get $O(1)$ · **put** $O(1)$ · space $O(\text{capacity})$. The map buys lookup; the doubly linked list buys removal without a scan.

ALTERNATIVE DESIGN

LinkedHashMap in access order with **removeEldestEntry** does this in five lines — but it hard-codes LRU, which is what the follow-up attacks.

FOLLOW-UPS TO EXPECT

- **"Make it most recently used instead."** The one this problem exists for. Behind an interface it is a new class; without one, a rewrite.
- **"Now LFU."** Harder — say plainly that the interface holds but the structure behind it does not.
- **"Make it thread-safe."** Name what you'd guard and what it costs.

COMMON MISTAKES

- A singly linked list — removal becomes $O(n)$ and the point is lost.
- Forgetting that **get** reorders. The most common correctness bug here.
- No key inside the node, so eviction cannot clean up the map.

INTERVIEWER'S PERSPECTIVE · REAL INTERVIEW STORY

[Placeholder — to be written from `interview-notes.md`, section B.]